RAMAIAH
Institute of Technology

# MINI PROJECT

## DIGITAL SIGNAL PROCESSING ET53

## Noise Removal Using Spectral Subtraction and NLP Techniques

**BACHELOR OF ENGINEERING IN**
**ELECTRONICS & TELECOMMUNICATION ENGINEERING**

**For the Course "Digital Signal Processing ET53"**
**Academic Year 2025-2026**

**Submitted by**

**Rishi Raj 1MS23ET069**

**Under the guidance of**
**Dr. Venu KN**

**Associate Professor,**
**Dept. of Electronics & Telecommunication Engineering**
**RIT, Bangalore-560054**

**RAMAIAH INSTITUTE OF TECHNOLOGY**
**(Autonomous Institute Affiliated to VTU)**
**Department of Electronics & Telecommunication Engineering**

**Bangalore-560054**

# INTRODUCTION

Noise removal is a key task in digital signal processing because real-world speech recordings often contain background sounds such as hums, traffic, or wind. These unwanted components reduce clarity and make it difficult for both humans and machines to interpret spoken content. As speech-based applications grow, like voice assistants, online communication, and transcription systems, cleaning audio before further processing becomes essential.

In this project, the goal is to enhance noisy speech using the **s**pectral subtraction method, a classical DSP technique. The approach transforms audio into the frequency domain using STFT, estimates the noise from an initial sample, subtracts it from the noisy spectrum, and reconstructs a cleaner version of the speech. Adjustments like over-subtraction and spectral flooring help reduce artifacts and preserve intelligibility.

After enhancement, the improved signal is transcribed using a Whisper-based NLP model. Whisper converts the cleaned audio into text, demonstrating the practical link between DSP preprocessing and NLP performance. The workflow includes loading audio, applying noise reduction, reconstructing the speech signal, and generating transcription.

This project follows classical DSP literature by implementing spectral subtraction for noise reduction and then applying a modern NLP transcription model. The combination reflects a widely used pipeline where traditional signal processing enhances audio quality, and NLP methods extract meaningful linguistic content. The literature strongly supports this hybrid approach for tasks where deep learning-based enhancement is unnecessary or computationally demanding.

## LITERATURE REVIEW

Research in speech enhancement has evolved from simple classical DSP techniques to more advanced adaptive and learning-based models. Early work mainly focused on analysing the spectral characteristics of noise and speech and applying mathematical operations to reduce unwanted components. One of the foundational methods in this area is spectral subtraction, where the noise spectrum is estimated from silent segments and subtracted from the noisy speech. Studies show that this method is effective for steady background noises and remains popular because it is easy to implement, computationally lightweight, and suitable for real-time applications. Variations such as over-subtraction, multi-band subtraction, and spectral flooring have been proposed to minimize artifacts like musical noise and improve perceptual quality.

Another significant line of research includes Wiener filtering, which computes an optimal filter using estimates of speech and noise power. It has been shown to work better than basic subtraction methods in many cases but depends heavily on accurate noise modelling.

Time–frequency analysis techniques such as the Short-Time Fourier Transform (STFT) form the backbone of most enhancement algorithms. STFT enables the signal to be analysed in short frames where speech and noise characteristics vary less, allowing more precise processing.

On the NLP side, automatic speech recognition (ASR) has transitioned from early template matching and HMM-based systems to deep learning-based architectures. Modern models like Whisper have been trained on large multi-lingual, noisy datasets, making them robust against many real-world distortions. Literature repeatedly emphasizes that applying noise reduction before ASR improves transcription accuracy, especially when dealing with low-quality recordings or environmental noise.

This project follows these well-established directions by implementing spectral subtraction for noise reduction and using Whisper for transcription. The combination aligns with research findings that enhancement through classical DSP helps improve the performance of NLP-driven speech recognition systems, making it a suitable approach for academic and practical applications.

# METHODOLOGY

**1. Audio Dataset / Input Collection**
The project uses recorded or downloaded speech audio that contains background noise such as humming, traffic, chatter, or wind.
Only single-speaker noisy speech samples are considered.
These audio files serve as input for both noise reduction and transcription tasks.

**2. Audio Preprocessing**
Each audio file undergoes basic preprocessing before applying DSP:

- Resampled to **16 kHz**
- Converted to **mono**
- Stored as a **NumPy array** for further processing
  This ensures consistent processing and compatibility with STFT and Whisper.

**3. Short-Time Fourier Transform (STFT)**
The audio is transformed from the time domain to the time–frequency domain using STFT.
STFT provides localized frequency information in frames, which is essential for noise estimation and spectral subtraction.
Parameters used:
Window size: **1024 samples**
Hop length: **512 samples**
Hann window
The STFT output is split into **magnitude** and **phase** components.

**4. Noise Estimation**
Noise is estimated from the first part of the signal, assuming it contains mostly background noise.
Steps:

- Select the first **0.9 seconds** of audio
- Compute the **average noise magnitude spectrum** across these frames
  This creates a noise profile that is used for subtraction.

**5. Noise Reduction (Spectral Subtraction)**
Manual spectral subtraction is applied using the following steps:

1. Subtract the noise magnitude from the noisy speech magnitude
2. Apply an **over-subtraction factor (α = 1.6)** to improve noise removal
3. Apply **spectral flooring** to reduce musical noise artifacts
4. Ensure no negative magnitude values appear after subtraction

This produces a cleaner magnitude spectrum while keeping the original phase.

**6. Signal Reconstruction (ISTFT)**
The enhanced magnitude is combined with the original phase and converted back to the time domain using the **Inverse STFT (ISTFT)**.
This generates the cleaned audio waveform, which is saved as `cleaned.wav`.

**7. Speech Transcription (NLP – Whisper Model)**
The cleaned audio is passed to the **Whisper Tiny model**, which performs automatic speech recognition.
Steps:

- Load the cleaned audio
- Run Whisper's transcription function
- Extract the recognized text

Whisper serves as the NLP component, converting speech content into readable text.

**8. Output Evaluation**
The system outputs:

- The cleaned audio waveform
- Before/after waveform comparison
- Transcribed text

Qualitative evaluation includes comparing:

- Original vs. cleaned audio clarity
- Transcription accuracy before and after noise removal

This demonstrates how DSP preprocessing improves the performance of speech-to-text NLP.

## Algorithm: Natural Scene Image Classification Using Machine Learning

START
↓
Audio Input Collection
(Record or upload noisy speech sample)
↓
Audio Preprocessing
• Resample to 16 kHz
• Convert to Mono
• Prepare Signal for STFT
↓
STFT Computation
• Convert Time Domain → Frequency Domain
• Extract Magnitude & Phase
↓
Noise Estimation
• Use first 0.5 seconds as noise sample
• Compute average noise spectrum
↓
Noise Reduction (Spectral Subtraction)
• Apply Over-Subtraction ($\alpha = 1.6$)
• Apply Spectral Flooring
• Obtain Clean Magnitude Spectrum
↓
Signal Reconstruction (ISTFT)
• Combine Clean Magnitude with Phase
• Convert Back to Time Domain
• Generate Cleaned Audio
↓
Speech Transcription (NLP)
• Use Whisper Model
• Recognize and Convert Speech to Text
↓
Output Evaluation
• Compare Original vs Cleaned Waveform
• Display Transcription Result
↓
END

# Code:

## 1. Importing Libraries

```
import librosa
import numpy as np
import soundfile as sf
```

- **librosa** → Used for loading audio, STFT, ISTFT, and audio-related DSP operations.
- **numpy** → Handles mathematical operations and array processing.
- **matplotlib** → Used to plot waveforms before and after noise reduction.
- **soundfile** → Saves the cleaned audio as a `.wav` file.

---

## 2. Loading the Audio File

```
y, sr = librosa.load(filename, sr=16000)
```

- `filename` → The audio file uploaded in Colab.
- `y` → The audio time-domain signal (samples).
  `sr=16000` → Resamples the audio to **16 kHz**, a standard sampling rate for speech.

---

## 3. STFT (Short-Time Fourier Transform)

```
D = librosa.stft(y)
magnitude, phase = np.abs(D), np.angle(D)
```

- **STFT** converts the signal from time domain → frequency domain.
- `D` → Complex-valued matrix representing frequencies across frames.
- `magnitude` → Strength of each frequency component.
- `phase` → Phase information needed for reconstructing the cleaned audio.

---

## 4. Noise Estimation

```
noise_frames = int(0.9 * sr / 512)
noise_mag = np.mean(magnitude[:, :noise_frames], axis=1,
keepdims=True)
```

- Assumes the **first 0.9 seconds** contains only background noise.
- Converts 0.9 seconds into number of STFT frames (`/512` = hop length).
- Computes the **average noise magnitude spectrum** over those frames.

---

## 5. Over-Subtraction Factor

```
alpha = 1.8
```

- Over-subtraction (α > 1) removes noise more aggressively.
- Higher values remove more noise but may distort speech slightly.

---

## 6. Spectral Subtraction

```
subtracted = magnitude - alpha * noise_mag
clean_mag = np.maximum(subtracted, 0.01 * noise_mag)
```

- `subtracted` = remove estimated noise from each frequency band.
- Uses over-subtraction to improve noise suppression.
- `np.maximum()` prevents negative values → called **spectral flooring**. Flooring (0.01 × noise) reduces musical noise artifacts.

---

## 7. Reconstructing Clean Audio

```
clean_D = clean_mag * np.exp(1j * phase)
clean_audio = librosa.istft(clean_D)
```

- Combines cleaned magnitude with original phase.
- Converts the signal back into time domain using ISTFT.
- Produces the final cleaned audio waveform.

---

## 8. Saving the Cleaned Audio

```
sf.write("cleaned.wav", clean_audio, sr)
print("Manual DSP noise removal complete")
```

- Writes the cleaned speech waveform to a .wav file.
- Allows playback, testing, and transcription.

---

## 9. Plotting Before & After Waveforms

```
plt.figure(figsize=(14,4))
plt.plot(y, label="Original")
plt.plot(clean_audio, label="Cleaned", alpha=0.7)
plt.legend()
plt.title("Before and After Noise Removal")
plt.show()
```

- Plots the original signal vs cleaned signal.
- Helps visually compare noise reduction effectiveness.
- Alpha=0.7 makes overlapping signals visible.

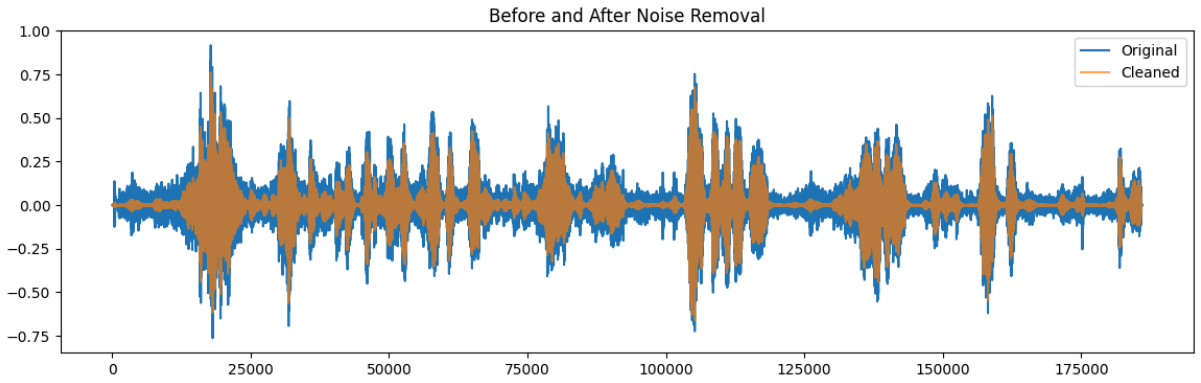
---

## 10. Speech Transcription (NLP Stage)

```
import whisper

model = whisper.load_model("tiny")
result = model.transcribe("cleaned.wav")
print(result["text"])
```

- Loads the **Whisper Tiny model** for speech recognition.
- Transcribes only the **cleaned audio**, resulting in better accuracy.
- Prints the recognized text (final output of NLP stage).




```
/usr/local/lib/python3.12/dist-packages/whisper/transcribe.py:132: UserWarning: FP16 is not supported on CPU; using FP32 instead
  warnings.warn("FP16 is not supported on CPU; using FP32 instead")

TRANSCRIPTION OF CLEANED AUDIO:
 Hello, I am speaking to the President of United States. Hello, hello. How is the weather there? It's rainy today. Awesome.
```

# Conclusion

In this project, we developed a system that enhances noisy speech using digital signal processing and then converts the cleaned audio into text using an NLP-based transcription model. The main focus was on applying the spectral subtraction method, a classical DSP technique, to reduce background noise by estimating a noise profile and subtracting it from the speech spectrum. After reconstruction, the cleaned audio showed noticeable improvement compared to the original noisy signal.

The enhanced audio was then passed to a Whisper speech recognition model, which produced clearer and more accurate transcription results. This demonstrated how preprocessing through DSP directly improves the performance of NLP systems. Overall, the project highlights the importance of combining signal processing with modern language models for real-world tasks such as transcription, communication enhancement, and audio analysis. It also provided a practical understanding of how frequency-domain processing and NLP techniques work together to handle noisy speech signals effectively.

## Future Scope

**1. Explore Advanced Noise Reduction Techniques**

The current system uses classical spectral subtraction. Future work can include Wiener filtering, MMSE estimators, or adaptive noise suppression to improve clarity and reduce artifacts.

**2. Improve Noise Estimation Methods**

Instead of relying on the first 0.5 seconds for noise estimation, dynamic noise tracking or machine-learning-based noise profiling can be used for environments where noise continuously changes.

**3. Enhance Speech Quality with Post-Processing**

Methods such as spectral smoothing, harmonic enhancement, or voice activity detection (VAD) can be added to reduce musical noise and further improve speech intelligibility.

**4. Use Larger Whisper Models for Better Accuracy**

Upgrading from the Tiny model to base or small versions can produce more accurate and robust transcription results, especially for long or complex speech recordings.

**5. Add Real-Time Implementation**

The system can be extended to process live microphone input and perform real-time noise reduction and transcription, making it suitable for online meetings and assistive communication tools.

**6. Build a Simple GUI or Web App**

A user-friendly interface can be created so users can upload an audio file, view noise reduction results, and see the transcription instantly. This makes the system easier to demonstrate and deploy.